Оглавление

Введение 2

Часть 1: Теоретические основы и математический аппарат 3

1.1. Базовые определения и постановка задачи 3

1.2. Семейство H-полиномов (Комплексный случай) 4

1.3. Матричная интерпретация 6

1.4. Связь с методом Ньютона-Рафсона 9

Часть 2. Трехэтапный алгоритм и анализ сходимости 11

2.1. Структура трех стадий 11

2.2. Доказательство глобальной сходимости 12

2.2. Доказательство глобальной сходимости 15

2.3. Скорость сходимости 16

2.4. Алгоритм для действительных полиномов (Real Polynomials) 17

Часть 3. Реализация, модификации и эффективность метода 19

3.1. Вычислительные аспекты и реализация (CPOLY/RPOLY) 19

3.2. Применение Быстрого Преобразования Фурье (FFT) 21

3.3. Принципы тестирования программ 22

Литература 24

Введение

Алгоритм Дженкинса-Трауба (Jenkins-Traub algorithm) представляет собой высокоэффективный итерационный метод численного нахождения нулей полиномов с комплексными коэффициентами. Разработанный Майклом А. Дженкинсом и Джозефом Ф. Траубом в 1970 году, этот метод стал стандартом де-факто в области математического программного обеспечения (например, он лежит в основе функции polyroot в библиотеке IMSL и реализации RPOLY).

Исторически алгоритм возник как развитие класса двухэтапных итерационных методов, предложенных Траубом в середине 1960-х годов. Основной целью разработки было создание процедуры, которая гарантирует глобальную сходимость (в отличие от метода Ньютона, требующего удачного начального приближения) и при этом обладает высокой скоростью сходимости (превосходящей квадратичную) на финальной стадии.

Ключевой особенностью алгоритма является трехэтапная структура, позволяющая преодолеть типичные проблемы численного поиска корней:

1. **Проблема кластеризации:** Плохая обусловленность при наличии кратных или близко расположенных корней.
2. **Проблема стабильности дефляции:** Накопление ошибок округления при последовательном делении полинома на найденные линейные множители.
3. **Проблема выбора начального приближения:** Необходимость автоматического выбора стартовой точки, гарантирующей сходимость.

Алгоритм существует в двух основных вариациях:

1. **CPOLY (Complex Polynomial Algorithm):** Общий случай для полиномов с комплексными коэффициентами. Работает в комплексной арифметике.
2. **RPOLY (Real Polynomial Algorithm):** Специализированная версия для полиномов с вещественными коэффициентами. Она избегает комплексной арифметики при поиске комплексно-сопряженных пар корней, работая с квадратичными множителями, что ускоряет вычисления примерно в 4 раза.

В данном отчете мы последовательно изложим теоретический фундамент метода, начиная с комплексного случая, так как он является концептуальной основой для более сложного вещественного алгоритма.

Часть 1: Теоретические основы и математический аппарат

В основе алгоритма лежит теория последовательностей полиномов, генерируемых специальными рекуррентными соотношениями. Мы введем понятие H-полиномов, которые играют роль, аналогичную последовательностям Штурма или результантам в других алгебраических методах, но адаптированную для итерационного поиска в комплексной плоскости.

1.1. Базовые определения и постановка задачи

Рассмотрим полином $P(z)$ степени n с комплексными коэффициентами: $P(z) = \sum_{i=0}^{n} a_i\, z^{n-i} = a_0 z^n + a_1 z^{n-1} + \cdots + a_n, \quad a_0 \neq 0, a_n \neq 0.$

Для удобства теоретического анализа (но не обязательно в реализации) часто полагают $a_0 = 1$.

Полином может быть представлен в факторизованном виде через свои нули: $P(z) = a_0 \prod_{i=1}^{j} (z - \rho_i)^{m_i}$,

где:

- $\rho_1, \rho_2, \dots, \rho_j$ — различные нули полинома.
- m_i — кратность нуля ρ_i.
- $\sum_{i=1}^{j} m_i = n$ — полная степень полинома.

Основная задача состоит в нахождении аппроксимаций нулей ρ_i и их кратностей. Алгоритм Дженкинса-Трауба находит нули по одному (или парами в вещественном случае) в порядке возрастания их модулей: $|\rho_1| \leq |\rho_2| \leq \cdots \leq |\rho_j|$. Порядок возрастания модулей критически важен для численной устойчивости процесса **дефляции** (deflation). После нахождения корня ρ_1 полином делится на $(z - \rho_1)$, и поиск продолжается для частного. Если удалять сначала большие по модулю корни, ошибки округления в коэффициентах оставшегося полинома могут катастрофически вырасти (эффект Уилкинсона). Нахождение малых корней первыми минимизирует этот эффект.

Определим **лагранжевы факторы** (или дефлированные полиномы) $P_i(z)$ для каждого корня ρ_i: $P_i(z) = \frac{P(z)}{z - \rho_i}$. Заметим, что $P_i(z)$ является полиномом степени

$n-1$. Производная исходного полинома $P'(z)$ может быть выражена через линейную комбинацию этих факторов: $P'(z) = \frac{d}{dz}\left(a_0 \prod_{k=1}^{j}(z-\rho_k)^{m_k}\right) = \sum_{i=1}^{j} m_i \frac{P(z)}{z-\rho_i} = \sum_{i=1}^{j} m_i P_i(z)$. Это разложение играет фундаментальную роль в построении алгоритма.

1.2. Семейство H-полиномов (Комплексный случай)

Центральным объектом теории Дженкинса-Трауба является последовательность полиномов $H^{(\lambda)}(z)$ степени $n-1$. Эти полиномы конструируются таким образом, чтобы "выделять" один из факторов $P_i(z)$ (соответствующий искомому корню) и подавлять остальные.

Рекуррентное определение

Последовательность $H^{(\lambda)}(z)$ определяется рекурсивно с помощью параметра сдвига s_λ. Пусть $H^{(0)}(z) = P'(z)$. Общий шаг рекурсии для перехода от λ к $\lambda+1$ при заданном сдвиге s_λ определяется формулой: $H^{(\lambda+1)}(z) = \frac{1}{z-s_\lambda}\left(H^{(\lambda)}(z) - \frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)}P(z)\right)$. Здесь предполагается, что $P(s_\lambda) \neq 0$.

Алгебраический смысл рекурсии:

Выражение в скобках сконструировано так, чтобы зануляться при $z = s_\lambda$. Действительно, подставим $z = s_\lambda$: $H^{(\lambda)}(s_\lambda) - \frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)}P(s_\lambda) = H^{(\lambda)}(s_\lambda) - H^{(\lambda)}(s_\lambda) = 0$. Поскольку выражение в скобках обращается в ноль в точке s_λ, оно (как полином) делится на $(z-s_\lambda)$ без остатка. Следовательно, $H^{(\lambda+1)}(z)$ действительно является полиномом, причем его степень на единицу меньше степени $P(z)$ (так как в скобках стоит комбинация полиномов степени $n-1$ и n, но член со старшей степенью z^n из $P(z)$ уничтожается при вычитании, если рассматривать приведенные формы, или корректируется коэффициентом). Точнее, $H^{(\lambda)}(z)$ имеет степень не выше $n-1$, $P(z)$ имеет степень n. Результат деления полинома степени n на линейный двучлен имеет степень $n-1$.

Явная формула для коэффициентов H-полиномов

Докажем фундаментальную лемму, раскрывающую структуру $H^{(\lambda)}(z)$ в терминах корней ρ_i. В оригинальных статьях это часто опускается со словами "легко проверить по индукции", но мы проведем полный вывод.

Лемма 1.

Если $H^{(0)}(z) = \sum_{i=1}^{j} c_i^{(0)} P_i(z)$, где $c_i^{(0)} = m_i$, то для любого λ полином $H^{(\lambda)}(z)$ имеет вид: $H^{(\lambda)}(z) = \sum_{i=1}^{j} c_i^{(\lambda)} P_i(z)$, где коэффициенты пересчитываются как: $c_i^{(\lambda+1)} = c_i^{(\lambda)} \cdot (s_\lambda - \rho_i)^{-1}$. В случае постоянного сдвига $s_\lambda \equiv s$ имеем: $c_i^{(\lambda)} = c_i^{(0)}(s - \rho_i)^{-\lambda}$.

Доказательство:

Проведем индукционный переход от λ к $\lambda + 1$. Предположим, что $H^{(\lambda)}(z) = \sum_{i=1}^{j} c_i^{(\lambda)} P_i(z)$. Подставим это выражение в рекуррентную формулу: $H^{(\lambda+1)}(z) = \frac{1}{z-s_\lambda}\left[\sum_{i=1}^{j} c_i^{(\lambda)} P_i(z) - \frac{\sum_{i=1}^{j} c_i^{(\lambda)} P_i(s_\lambda)}{P(s_\lambda)} P(z)\right]$. Вспомним определение $P_i(z) = \frac{P(z)}{z-\rho_i}$. Подставим его значения в точках z и s_λ: $P_i(z) = \frac{P(z)}{z-\rho_i}$, $P_i(s_\lambda) = \frac{P(s_\lambda)}{s_\lambda-\rho_i}$. Заменим $P_i(s_\lambda)$ во втором слагаемом внутри скобок: $\frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)} P(z) = \frac{1}{P(s_\lambda)}\left(\sum_{i=1}^{j} c_i^{(\lambda)} \frac{P(s_\lambda)}{s_\lambda-\rho_i}\right) P(z) = \left(\sum_{i=1}^{j} \frac{c_i^{(\lambda)}}{s_\lambda-\rho_i}\right) P(z)$. Теперь рассмотрим выражение для $H^{(\lambda+1)}(z)$ целиком, вынося общий множитель $P(z)$: $H^{(\lambda+1)}(z) = \frac{1}{z-s_\lambda}\left[\sum_{i=1}^{j} c_i^{(\lambda)} \frac{P(z)}{z-\rho_i} - \sum_{i=1}^{j} c_i^{(\lambda)} \frac{P(z)}{s_\lambda-\rho_i}\right]$. Объединим суммы: $H^{(\lambda+1)}(z) = \frac{P(z)}{z-s_\lambda} \sum_{i=1}^{j} c_i^{(\lambda)} \left(\frac{1}{z-\rho_i} - \frac{1}{s_\lambda-\rho_i}\right)$. Преобразуем выражение в скобках (разность дробей): $\frac{1}{z-\rho_i} - \frac{1}{s_\lambda-\rho_i} = \frac{(s_\lambda-\rho_i)-(z-\rho_i)}{(z-\rho_i)(s_\lambda-\rho_i)} = \frac{s_\lambda-z}{(z-\rho_i)(s_\lambda-\rho_i)} = -\frac{z-s_\lambda}{(z-\rho_i)(s_\lambda-\rho_i)}$. Подставляем полученную разность обратно в формулу для $H^{(\lambda+1)}(z)$: $H^{(\lambda+1)}(z) = \frac{P(z)}{z-s_\lambda} \sum_{i=1}^{j} c_i^{(\lambda)} \left[-\frac{z-s_\lambda}{(z-\rho_i)(s_\lambda-\rho_i)}\right]$. Множитель $(z - s_\lambda)$ в числителе и знаменателе сокращается: $H^{(\lambda+1)}(z) = P(z) \sum_{i=1}^{j} c_i^{(\lambda)} \frac{-1}{(z-\rho_i)(s_\lambda-\rho_i)}$. Учтем знак: в формуле рекурсии знаменатель был $(z - s_\lambda)$, разность дала $(s_\lambda - z) = -(z - s_\lambda)$. Однако, при переходе к коэффициентам c_i, нам нужно выражение вида $\sum \tilde{c}_i P_i(z) = \sum \tilde{c}_i \frac{P(z)}{z-\rho_i}$. Перепишем предыдущее равенство: $H^{(\lambda+1)}(z) = \sum_{i=1}^{j} \left[c_i^{(\lambda)} \frac{-1}{s_\lambda-\rho_i}\right] \frac{P(z)}{z-\rho_i}$. Заметим, что $\frac{-1}{s_\lambda-\rho_i} = \frac{1}{\rho_i-s_\lambda}$. Однако, формулировка Леммы 1 утверждает $c_i^{(\lambda+1)} = c_i^{(\lambda)}(s_\lambda - \rho_i)^{-1}$. Проверим знак. В исходной статье Дженкинса-Трауба (1970) используется определение $P_i(z) = P(z)/(z - \rho_i)$. Разность $\frac{1}{z-\rho} - \frac{1}{s-\rho} = \frac{s-z}{(z-\rho)(s-\rho)}$. Делим на $z - s$: получаем $\frac{-1}{(z-\rho)(s-\rho)}$.

Тогда коэффициент при $P_i(z)$ будет $c_i^{(\lambda)} \frac{-1}{s_\lambda - \rho_i} = c_i^{(\lambda)} \frac{1}{\rho_i - s_\lambda}$. В ряде источников (например, в оригинале 1970 г., формула 2.4) указано: $c_i^{(\lambda)} = c_i^{(0)} \prod (\rho_i - s_t)^{-1}$. Это согласуется с нашим выводом. Если же использовать определение $(s_\lambda - \rho_i)^{-1}$, знак будет противоположным. Уточнение по источнику [Jenkins-Traub 1970, Eq 2.4]: там стоит $\sigma_i^{-\lambda}$ (для вещественного случая квадратичных факторов) или $(\rho_i - s)^{-1}$ для линейных. Примем соглашение: $c_i^{(\lambda+1)} = \frac{c_i^{(\lambda)}}{\rho_i - s_\lambda}$ (или эквивалентно $-(s_\lambda - \rho_i)^{-1}$). Суть в том, что веса c_i изменяются обратно пропорционально расстоянию от сдвига s_λ до корня ρ_i. Если s_λ близко к ρ_1, то знаменатель $(\rho_1 - s_\lambda)$ мал, и коэффициент $c_1^{(\lambda+1)}$ растет намного быстрее остальных. Это обеспечивает доминирование члена, содержащего $P_1(z)$, в сумме $H^{(\lambda+1)}(z)$.

Нормированная форма

На практике коэффициенты H-полиномов могут быстро расти или убывать, вызывая переполнение или исчезновение порядка. Поэтому вводится **нормированный H-полином**: $\bar{H}^{(\lambda)}(z) = \frac{H^{(\lambda)}(z)}{lc(H^{(\lambda)})}$, где $lc(\cdot)$ обозначает старший коэффициент (leading coefficient). Нормированный полином $\bar{H}^{(\lambda)}(z)$ при сходимости процесса стремится к полиному $P_1(z) = \frac{P(z)}{z - \rho_1}$, у которого старший коэффициент равен a_0.

Следующий раздел будет посвящен матричной интерпретации этого процесса и связи с методом Ньютона.

1.3. Матричная интерпретация

Алгоритм Дженкинса-Трауба имеет глубокую связь с методами линейной алгебры, в частности с обратной итерацией (inverse powering) и обобщенной итерацией Рэлея (Generalized Rayleigh Iteration). Эта интерпретация позволяет взглянуть на процесс поиска нулей полинома как на поиск собственных значений специально сконструированной матрицы.

Сопровождающая матрица

Рассмотрим полином $P(z) = z^n + a_1 z^{n-1} + \cdots + a_n$ с ведущим коэффициентом $a_0 = 1$. Ему соответствует **сопровождающая матрица** (Companion Matrix) A размера $n \times n$:

$$A = \begin{pmatrix} 0 & 0 & \dots & 0 & -a_n \\ 1 & 0 & \dots & 0 & -a_{n-1} \\ 0 & 1 & \dots & 0 & -a_{n-2} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 1 & -a_1 \end{pmatrix}.$$

Характеристический полином этой матрицы совпадает с $P(z)$ (с точностью до знака при нечетном n): $\det(zI - A) = P(z)$. Следовательно, собственные значения матрицы A в точности равны нулям $\rho_1, \dots, \rho_n$ полинома $P(z)$.

Обозначим правые собственные векторы матрицы A, соответствующие собственным значениям ρ_i, как $\underline{p}_i$, а левые собственные векторы как $\underline{q}_i^T$. Компоненты этих векторов имеют специфическую структуру, связанную с полиномами.

Правый собственный вектор $\underline{p}_i$: $\underline{p}_i = \begin{pmatrix} p_{i,n-1} \\ \vdots \\ p_{i,0} \end{pmatrix}$, где $p_{i,j}$ — коэффициенты лагранжева фактора $P_i(z) = \frac{P(z)}{z-\rho_i} = \sum_{j=0}^{n-1} p_{i,j}\, z^{n-1-j}$.

Левый собственный вектор $\underline{q}_i^T$: $\underline{q}_i^T = (1, \rho_i, \rho_i^2, \dots, \rho_i^{n-1})$. Действительно, проверим условие левого собственного вектора: $\underline{q}_i^T A = (\rho_i, \rho_i^2, \dots, \rho_i^{n-1}, -a_n - a_{n-1}\rho_i - \dots - a_1\rho_i^{n-1})$.

Последняя компонента вектора равна: $-a_n - a_{n-1}\rho_i - \dots - a_1\rho_i^{n-1} = \rho_i^n - P(\rho_i) = \rho_i^n$, так как $P(\rho_i) = 0$. Таким образом, $\underline{q}_i^T A = \rho_i(1, \rho_i, \dots, \rho_i^{n-1}) = \rho_i \underline{q}_i^T$, что и требовалось доказать.

<u>Связь H-полиномов с обратной итерацией</u>

Полином $H^{(\lambda)}(z) = \sum_{k=0}^{n-1} h_k^{(\lambda)}\, z^{n-1-k}$ можно взаимно однозначно сопоставить с вектором его коэффициентов $\underline{h}^{(\lambda)} = (h_{n-1}^{(\lambda)}, \dots, h_0^{(\lambda)})^T$. Оказывается, что рекуррентное соотношение для $H^{(\lambda)}(z)$ в векторной форме эквивалентно методу обратной итерации со сдвигом (shifted inverse iteration) для матрицы A.

Теорема 1 (Матричная форма рекурсии).

Рекуррентное соотношение для H-полиномов с переменным сдвигом s_λ $H^{(\lambda+1)}(z) = \frac{1}{z-s_\lambda}\left(H^{(\lambda)}(z) - \frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)} P(z)\right)$ эквивалентно матричному уравнению: $\underline{h}^{(\lambda+1)} = (A - s_\lambda I)^{-1} \underline{h}^{(\lambda)}$.

Доказательство:

Рассмотрим уравнение $(A - s_\lambda I)\underline{h}^{(\lambda+1)} = \underline{h}^{(\lambda)}$. Левая часть представляет собой умножение матрицы $(A - s_\lambda I)$ на вектор коэффициентов полинома $H^{(\lambda+1)}(z)$. В терминах полиномов умножение на сопровождающую матрицу A соответствует умножению на переменную z по модулю $P(z)$. То есть, вектору $A\underline{v}$ соответствует полином $(zV(z)) \pmod{P(z)}$. Соответственно, вектору $(A - s_\lambda I)\underline{h}^{(\lambda+1)}$ соответствует полином: $((z - s_\lambda)H^{(\lambda+1)}(z)) \pmod{P(z)}$. Из рекуррентного определения $H^{(\lambda+1)}(z)$ мы знаем, что: $(z - s_\lambda)H^{(\lambda+1)}(z) = H^{(\lambda)}(z) - \frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)}P(z)$. Взяв это выражение по модулю $P(z)$, получаем: $((z - s_\lambda)H^{(\lambda+1)}(z)) \pmod{P(z)} = H^{(\lambda)}(z) \pmod{P(z)}$. Поскольку степень $H^{(\lambda)}(z)$ меньше n, $H^{(\lambda)}(z) \pmod{P(z)} = H^{(\lambda)}(z)$. Следовательно, полином, соответствующий вектору левой части, равен $H^{(\lambda)}(z)$. Это доказывает эквивалентность полиномиальной и матричной форм.

Обобщенная итерация Рэлея

Третья стадия алгоритма Дженкинса-Трауба использует переменный сдвиг $s_{\lambda+1}$, вычисляемый по формуле: $s_{\lambda+1} = s_\lambda - \frac{P(s_\lambda)}{\bar{H}^{(\lambda+1)}(s_\lambda)}$. Покажем, что это эквивалентно вычислению **обобщенного отношения Рэлея** (Generalized Rayleigh Quotient).

Введем вектор $\underline{s}^{(\lambda)} = (1, s_\lambda, s_\lambda^2, \dots, s_\lambda^{n-1})^T$. Заметим, что значение полинома $H(z)$ в точке s_λ можно записать как скалярное произведение: $H(s_\lambda) = \sum_{k=0}^{n-1} h_k\, s_\lambda^{n-1-k} = (\underline{s}^{(\lambda)})^T\underline{h}$, (с точностью до порядка индексов, в оригинальной статье используется обозначение $[\underline{s}^{(\lambda)}]^T\underline{h}^{(\lambda+1)} = H^{(\lambda+1)}(s_\lambda)$).

Формулу обновления сдвига можно переписать в виде: $s_{\lambda+1} = \frac{(\underline{s}^{(\lambda)})^T A\underline{h}^{(\lambda+1)}}{(\underline{s}^{(\lambda)})^T\underline{h}^{(\lambda+1)}}$.

Доказательство:

Выражение для обновления сдвига можно преобразовать: $s_{\lambda+1} = s_\lambda - \frac{P(s_\lambda)}{\bar{H}^{(\lambda+1)}(s_\lambda)} = s_\lambda - \frac{P(s_\lambda)\cdot \mathrm{lc}(H^{(\lambda+1)})}{H^{(\lambda+1)}(s_\lambda)}$. Здесь $\mathrm{lc}(H)$ — старший коэффициент. В оригинальной статье используется нормировка, где $\bar{H}$ уже делен на старший коэффициент, но для матричной формы удобнее работать с ненормированными величинами. Заметим, что из рекурсии следует: $h_0^{(\lambda+1)} = -\frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)}$ (коэффициент при z^{n-1}).

Подставим это в формулу сдвига: $s_{\lambda+1} = s_\lambda + \frac{P(s_\lambda)h_0^{(\lambda+1)}}{H^{(\lambda+1)}(s_\lambda)} = \frac{s_\lambda H^{(\lambda+1)}(s_\lambda)+P(s_\lambda)h_0^{(\lambda+1)}}{H^{(\lambda+1)}(s_\lambda)}$. Рассмотрим числитель. Из рекуррентного соотношения: $(z - s_\lambda)H^{(\lambda+1)}(z) = H^{(\lambda)}(z) - \frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)}P(z) = H^{(\lambda)}(z) + h_0^{(\lambda+1)}P(z)$. Перепишем это как: $zH^{(\lambda+1)}(z) - s_\lambda H^{(\lambda+1)}(z) - h_0^{(\lambda+1)}P(z) = H^{(\lambda)}(z)$. Или: $s_\lambda H^{(\lambda+1)}(s_\lambda) + h_0^{(\lambda+1)}P(s_\lambda) = zH^{(\lambda+1)}(z) - H^{(\lambda)}(z)$. Вычислим значение в точке s_λ. Это не дает прямой подстановки в числитель дроби Рэлея. Однако, вернемся к матричной форме. Числитель отношения Рэлея $(\underline{s}^{(\lambda)})^T A\underline{h}^{(\lambda+1)}$. Вектор $A\underline{h}^{(\lambda+1)}$ соответствует полиному $(zH^{(\lambda+1)}(z)) \pmod{P(z)}$. Обозначим этот полином $G(z)$. Тогда $(\underline{s}^{(\lambda)})^T A\underline{h}^{(\lambda+1)} = G(s_\lambda)$. Так как $zH^{(\lambda+1)}(z) = G(z) + Q(z)P(z)$, где $Q(z)$ — частное (в данном случае константа, равная старшему коэффициенту $H^{(\lambda+1)}$, то есть $h_0^{(\lambda+1)}$), то: $G(s_\lambda) = s_\lambda H^{(\lambda+1)}(s_\lambda) - h_0^{(\lambda+1)}P(s_\lambda)$. Таким образом, формула обновления сдвига действительно является обобщенной итерацией Рэлея. Вектор $\underline{h}^{(\lambda)}$ сходится к правому собственному вектору $\underline{p}_1$, а вектор $\underline{s}^{(\lambda)}$ сходится к левому собственному вектору $\underline{q}_1$ (поскольку $s_\lambda \to \rho_1$, то $(1, s_\lambda, \dots) \to (1, \rho_1, \dots)$).

Это объясняет высокую скорость сходимости алгоритма: итерация Рэлея для несимметричных матриц имеет кубическую сходимость (в общем случае) или квадратичную при кратных корнях, но в данном специальном случае (структура сопровождающей матрицы) удается избежать многих проблем общей итерации Рэлея.

1.4. Связь с методом Ньютона-Рафсона

Третья стадия алгоритма Дженкинса-Трауба может быть интерпретирована как применение метода Ньютона-Рафсона к специальной последовательности рациональных функций. Это дает интуитивное понимание механизма сходимости.

Определим последовательность рациональных функций $W^{(\lambda)}(z)$: $W^{(\lambda)}(z) = \frac{P(z)}{H^{(\lambda)}(z)}$.

Теорема 2 (Эквивалентность Ньютону).

Итерационная формула третьей стадии: $s_{\lambda+1} = s_\lambda - \frac{P(s_\lambda)}{\bar{H}^{(\lambda+1)}(s_\lambda)}$ тождественно совпадает с шагом метода Ньютона, примененным к функции $W^{(\lambda)}(z)$ в точке s_λ: $s_{\lambda+1} = s_\lambda - \frac{W^{(\lambda)}(s_\lambda)}{[W^{(\lambda)}(s_\lambda)]'}$.

Доказательство:

Введем вспомогательную функцию $V^{(\lambda)}(z) = \frac{H^{(\lambda)}(z)}{P(z)} = \frac{1}{W^{(\lambda)}(z)}$. Рекуррентное соотношение для H-полиномов можно переписать в терминах V: $V^{(\lambda+1)}(z) = \frac{H^{(\lambda+1)}(z)}{P(z)} = \frac{1}{z-s_\lambda}\left(\frac{H^{(\lambda)}(z)}{P(z)} - \frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)}\right) = \frac{V^{(\lambda)}(z)-V^{(\lambda)}(s_\lambda)}{z-s_\lambda}$.

Заметим, что предел этого выражения при $z \to s_\lambda$ есть производная: $\lim_{z\to s_\lambda} V^{(\lambda+1)}(z) = [V^{(\lambda)}(s_\lambda)]'$. Однако $V^{(\lambda+1)}(z)$ — рациональная функция, и ее значение в точке s_λ формально определено как этот предел (устранимая особенность). В оригинальной работе показано, что нормированный полином $\bar{H}^{(\lambda+1)}$ удовлетворяет соотношению: $\bar{H}^{(\lambda+1)}(s_\lambda) \sim [V^{(\lambda)}(s_\lambda)]'$. Точнее: $s_{\lambda+1} = s_\lambda + \frac{V^{(\lambda)}(s_\lambda)}{[V^{(\lambda)}(s_\lambda)]'}$. Так как $W(z) = 1/V(z)$, то $W'(z) = -V'(z)/V^2(z)$. Следовательно, отношение Ньютона: $\frac{W(s)}{W'(s)} = \frac{1/V(s)}{-V'(s)/V^2(s)} = -\frac{V(s)}{V'(s)}$. Тогда шаг Ньютона: $s - \frac{W(s)}{W'(s)} = s + \frac{V(s)}{V'(s)}$. Это совпадает с формулой обновления сдвига.

Физический смысл:

Мы применяем метод Ньютона не к фиксированной функции $P(z)$, а к последовательности функций $P(z)/H^{(\lambda)}(z)$. Поскольку $H^{(\lambda)}(z)$ сходится к $P(z)/(z-\rho_1)$ (с точностью до константы), то отношение $P(z)/H^{(\lambda)}(z)$ сходится к линейной функции $C(z-\rho_1)$. Метод Ньютона для линейной функции сходится за одну итерацию. Таким образом, алгоритм Дженкинса-Трауба можно рассматривать как метод Ньютона, применяемый к функции, которая сама "исправляется" на каждом шаге, становясь все более похожей на линейную функцию в окрестности корня. Это объясняет его сверхбыструю сходимость.

Часть 2. Трехэтапный алгоритм и анализ сходимости

Алгоритм Дженкинса-Трауба состоит из трех последовательных стадий, каждая из которых использует итерацию, порождающую последовательность H-полиномов. Целью является генерация последовательности, которая на третьей стадии быстро сходится к одному из лагранжевых факторов $P_1(z) = P(z)/(z - \rho_1)$, что эквивалентно нахождению корня ρ_1.

2.1. Структура трех стадий

Стадия 1: Процесс без сдвига (No-Shift Process)

На этой стадии сдвиг s_λ полагается равным нулю для всех итераций. Пусть $H^{(0)}(z) = P'(z)$. Для $\lambda = 0,1,\dots,M-1$ вычисляется: $H^{(\lambda+1)}(z) = \frac{1}{z}\left(H^{(\lambda)}(z) - \frac{H^{(\lambda)}(0)}{P(0)}P(z)\right)$. Эта формула применима, если $P(0) \neq 0$. Если $P(0) = 0$, то $z = 0$ является корнем, который можно немедленно удалить дефляцией.

Математический смысл:

Согласно Лемме 1 (выведенной в Части 1), коэффициенты $c_i^{(\lambda)}$ в разложении $H^{(\lambda)}(z) = \sum c_i^{(\lambda)} P_i(z)$ преобразуются по закону: $c_i^{(\lambda)} = c_i^{(0)}(0 - \rho_i)^{-\lambda} = c_i^{(0)}(-\rho_i)^{-\lambda}$. Отношение коэффициентов для двух различных корней ρ_i и ρ_j меняется как: $\frac{c_i^{(\lambda)}}{c_j^{(\lambda)}} = \frac{c_i^{(0)}}{c_j^{(0)}}\left(\frac{\rho_j}{\rho_i}\right)^\lambda$. Если $|\rho_i| < |\rho_j|$, то это отношение растет по модулю. Таким образом, стадия без сдвига служит для **акцентирования** (accentuation) малых по модулю нулей. Члены суммы, соответствующие самым малым корням, становятся доминирующими. Обычно M выбирается небольшим (например, $M = 5$), так как эта стадия является лишь подготовительной.

Стадия 2: Процесс с фиксированным сдвигом (Fixed-Shift Process)

На этой стадии выбирается фиксированный параметр сдвига s, такой что $|s| = \beta$, где β — положительное число, не превышающее модуль самого малого корня. Сдвиг выбирается случайным образом на окружности радиуса β, чтобы избежать точного совпадения с каким-либо корнем. Условие выбора s: $|s - \rho_1| < |s - \rho_i|$ для всех $i = 2,\dots,j$. То есть s ближе к корню ρ_1, чем к любому другому.

Для $\lambda = M, M+1, \dots, L-1$ вычисляется: $H^{(\lambda+1)}(z) = \frac{1}{z-s}\left(H^{(\lambda)}(z) - \frac{H^{(\lambda)}(s)}{P(s)}P(z)\right)$.

Математический смысл:

Коэффициенты преобразуются как: $c_i^{(\lambda)} = c_i^{(M)}(s-\rho_i)^{-(\lambda-M)}$. Отношение коэффициентов: $\frac{c_i^{(\lambda)}}{c_1^{(\lambda)}} = \frac{c_i^{(M)}}{c_1^{(M)}}\left(\frac{s-\rho_1}{s-\rho_i}\right)^{\lambda-M}$. Поскольку $|s-\rho_1| < |s-\rho_i|$, дробь в скобках по модулю меньше единицы. С ростом λ это отношение стремится к нулю. Полином $H^{(\lambda)}(z)$ начинает сходиться (по направлению) к $P_1(z)$. Стадия продолжается до тех пор, пока не будет выполнен тест на сходимость (описан ниже), либо пока не будет превышен лимит итераций (в этом случае выбирается новый случайный сдвиг s).

<u>Стадия 3: Процесс с переменным сдвигом (Variable-Shift Process)</u>

Это основная стадия, обеспечивающая сверхбыструю сходимость. Сдвиг s_λ обновляется на каждом шаге. Начальное значение s_L берется из последней итерации второй стадии (как оценка корня). Для $\lambda = L, L+1, \dots$ вычисляется: $H^{(\lambda+1)}(z) = \frac{1}{z-s_\lambda}\left(H^{(\lambda)}(z) - \frac{H^{(\lambda)}(s_\lambda)}{P(s_\lambda)}P(z)\right)$, $s_{\lambda+1} = s_\lambda - \frac{P(s_\lambda)}{\bar{H}^{(\lambda+1)}(s_\lambda)}$, где $\bar{H}^{(\lambda+1)}(z) = H^{(\lambda+1)}(z)/\mathrm{lc}(H^{(\lambda+1)})$, а $\mathrm{lc}(\cdot)$ — старший коэффициент.

2.2. Доказательство глобальной сходимости

Анализ сходимости разделяется на две части: доказательство того, что третья стадия сходится при достаточно хорошем начальном приближении, и доказательство того, что первые две стадии гарантированно предоставляют такое приближение.

Рассмотрим **Теорему 3.1** из оригинальной работы [Jenkins-Traub 1970], определяющую условия сходимости третьей стадии.

Пусть $R = \min_{i=2,\dots,j}|\rho_1 - \rho_i|$ — минимальное расстояние от ρ_1 до остальных корней. Введем обозначения для относительных весов коэффициентов в разложении $H^{(\lambda)}(z)$: $d_i^{(\lambda)} = \frac{c_i^{(\lambda)}}{c_1^{(\lambda)}}$. Определим "ошибку сходимости" T_λ как отношение ошибок на соседних шагах: $T_\lambda = \left|\frac{s_{\lambda+1}-\rho_1}{s_\lambda-\rho_1}\right|$.

Теорема 3.1.

Предположим, что выполнены следующие условия:

1. $|s_L - \rho_1| < \frac{1}{2}R$.
2. $c_1^{(L)} \neq 0$.
3. $D_L = \sum_{i=2}^{j} |\, d_i^{(L)}| < \frac{1}{3}$.

Тогда последовательность сдвигов s_λ сходится к корню ρ_1 при $\lambda \to \infty$.

Доказательство:

<u>*Шаг 1. Вывод уравнения ошибки.*</u>

Сначала получим явное выражение для $s_{\lambda+1} - \rho_1$. Используем формулу обновления сдвига: $s_{\lambda+1} = s_\lambda - \frac{P(s_\lambda)}{\bar{H}^{(\lambda+1)}(s_\lambda)}$. Здесь $\bar{H}$ — нормированный полином. Заметим, что в формуле Ньютона $P(s)/\bar{H}(s)$ нормировка $\bar{H}$ важна. Обозначим $\mathcal{K}_\lambda = \sum_{i=1}^{j} c_i^{(\lambda+1)}$. Это сумма коэффициентов в разложении $H^{(\lambda+1)}$ по базису $P_i(z)$, которая является старшим коэффициентом $H^{(\lambda+1)}(z)$, так как старшие коэффициенты всех $P_i(z)$ равны 1 (при $a_0 = 1$). Тогда $\bar{H}^{(\lambda+1)}(z) = \frac{1}{\mathcal{K}_\lambda}\sum_{i=1}^{j} c_i^{(\lambda+1)} P_i(z)$. Подставим это в формулу сдвига: $s_{\lambda+1} = s_\lambda - \frac{P(s_\lambda)\mathcal{K}_\lambda}{\sum_{i=1}^{j} c_i^{(\lambda+1)} P_i(s_\lambda)}$. Используем свойство $P_i(s_\lambda) = \frac{P(s_\lambda)}{s_\lambda - \rho_i}$. Подставим в знаменатель: $\sum_{i=1}^{j} c_i^{(\lambda+1)} P_i(s_\lambda) = P(s_\lambda)\sum_{i=1}^{j} \frac{c_i^{(\lambda+1)}}{s_\lambda - \rho_i}$. $P(s_\lambda)$ сокращается: $s_{\lambda+1} = s_\lambda - \frac{\mathcal{K}_\lambda}{\sum_{i=1}^{j} \frac{c_i^{(\lambda+1)}}{s_\lambda - \rho_i}}$. Вычтем ρ_1 из обеих частей: $s_{\lambda+1} - \rho_1 = (s_\lambda - \rho_1) - \frac{\sum_{k=1}^{j} c_k^{(\lambda+1)}}{\sum_{i=1}^{j} \frac{c_i^{(\lambda+1)}}{s_\lambda - \rho_i}}$. Приведем к общему знаменателю: $s_{\lambda+1} - \rho_1 = \frac{(s_\lambda - \rho_1)\sum_{i=1}^{j} \frac{c_i^{(\lambda+1)}}{s_\lambda - \rho_i} - \sum_{k=1}^{j} c_k^{(\lambda+1)}}{\sum_{i=1}^{j} \frac{c_i^{(\lambda+1)}}{s_\lambda - \rho_i}}$. Рассмотрим числитель. Разобьем суммы на слагаемое с $i = 1$ и остальные ($i \geq 2$). Первое слагаемое суммы в знаменателе (умноженное на $s_\lambda - \rho_1$): $(s_\lambda - \rho_1)\frac{c_1^{(\lambda+1)}}{s_\lambda - \rho_1} = c_1^{(\lambda+1)}$. Этот член сокращается с $c_1^{(\lambda+1)}$ из второй суммы (где $k = 1$). Остается: Числитель $= \sum_{i=2}^{j} c_i^{(\lambda+1)} \frac{s_\lambda - \rho_1}{s_\lambda - \rho_i} - \sum_{i=2}^{j} c_i^{(\lambda+1)} = \sum_{i=2}^{j} c_i^{(\lambda+1)} \left(\frac{s_\lambda - \rho_1}{s_\lambda - \rho_i} - 1\right)$. Преобразуем скобку: $\frac{s_\lambda - \rho_1}{s_\lambda - \rho_i} - 1 = \frac{s_\lambda - \rho_1 - (s_\lambda - \rho_i)}{s_\lambda - \rho_i} = \frac{\rho_i - \rho_1}{s_\lambda - \rho_i}$. Таким образом: $s_{\lambda+1} -$

$\rho_1 = \frac{\sum_{i=2}^{j} c_i^{(\lambda+1)} \frac{\rho_i - \rho_1}{s_\lambda - \rho_i}}{\sum_{i=1}^{j} \frac{c_i^{(\lambda+1)}}{s_\lambda - \rho_i}}$. Разделим числитель и знаменатель на $\frac{c_1^{(\lambda+1)}}{s_\lambda - \rho_1}$. В знаменателе первый член ($i = 1$) превратится в 1. Введем обозначения: $r_i^{(\lambda)} = \frac{s_\lambda - \rho_1}{s_\lambda - \rho_i}$, $d_i^{(\lambda+1)} = \frac{c_i^{(\lambda+1)}}{c_1^{(\lambda+1)}}$. В оригинальной статье используются величины $d_i^{(\lambda)}$, которые пересчитываются как $d_i^{(\lambda+1)} = d_i^{(\lambda)} r_i^{(\lambda)}$. (Действительно, из Леммы 1: $c_i^{(\lambda+1)} \propto c_i^{(\lambda)}/(s_\lambda - \rho_i)$, следовательно $\frac{c_i^{(\lambda+1)}}{c_1^{(\lambda+1)}} = \frac{c_i^{(\lambda)}}{c_1^{(\lambda)}} \frac{s_\lambda - \rho_1}{s_\lambda - \rho_i}$).

После преобразований получаем фундаментальное уравнение эволюции ошибки (Eq 3.2 в статье): $\frac{s_{\lambda+1} - \rho_1}{s_\lambda - \rho_1} = \frac{\sum_{i=2}^{j} d_i^{(\lambda+1)} (r_i^{(\lambda)} - 1)/r_i^{(\lambda)}}{1 + \sum_{i=2}^{j} d_i^{(\lambda+1)}/r_i^{(\lambda)}}$.

Примечание: В статье [Jenkins-Traub 1970] уравнение (3.2) приведено в несколько ином виде через квадраты r_i, но суть та же. Восстановим точный вид из статьи. В статье [J-T 1970] уравнение (3.2) записано так: $\frac{s_{\lambda+1} - \rho_1}{s_\lambda - \rho_1} = \frac{\sum_{i=2}^{j} [r_i^{(\lambda)}]^2 d_i^{(\lambda)} - \sum_{i=2}^{j} r_i^{(\lambda)} d_i^{(\lambda)}}{1 + \sum_{i=2}^{j} [r_i^{(\lambda)}]^2 d_i^{(\lambda)}}$. Это получается, если выразить $d_i^{(\lambda+1)}$ через $d_i^{(\lambda)}$ и $r_i^{(\lambda)}$.

<u>Шаг 2. Оценка по индукции.</u>

Нам нужно доказать, что $T_\lambda = \left| \frac{s_{\lambda+1} - \rho_1}{s_\lambda - \rho_1} \right| \le \tau_L < 1$ для всех $\lambda \ge L$.

База индукции ($\lambda = L$):

Из условия (i) $|s_L - \rho_1| < \frac{1}{2} R$ и определения R, имеем для $i \ge 2$: $|s_L - \rho_i| \ge |\rho_1 - \rho_i| - |s_L - \rho_1| > R - \frac{1}{2} R = \frac{1}{2} R$. Следовательно, $|r_i^{(L)}| = \frac{|s_L - \rho_1|}{|s_L - \rho_i|} < \frac{R/2}{R/2} = 1$. Оценим числитель и знаменатель в формуле ошибки. Числитель ограничен сверху: $\sum |d_i^{(L)}| |r_i^{(L)}| (|r_i^{(L)}| + 1) < 2 \sum |d_i^{(L)}| = 2D_L$. Знаменатель ограничен снизу: $1 - \sum |d_i^{(L)}| |r_i^{(L)}|^2 > 1 - \sum |d_i^{(L)}| = 1 - D_L$. Таким образом: $T_L \le \frac{2D_L}{1 - D_L}$. Обозначим эту величину через τ_L. Условие (iii) $D_L < 1/3$ гарантирует, что: $\tau_L < \frac{2(1/3)}{1 - 1/3} = \frac{2/3}{2/3} = 1$. Следовательно, шаг сжатия выполнен: ошибка уменьшилась.

Шаг индукции:

Предположим, что $T_k \le \tau_L < 1$ для всех $k = L, \dots, \lambda - 1$. Это означает, что сдвиги s_k остаются в круге радиуса $R/2$ вокруг ρ_1, и даже приближаются к нему. Следовательно, условия $|r_i^{(k)}| < 1$ сохраняются. Кроме того, $d_i^{(\lambda)} = d_i^{(L)} \prod_{k=L}^{\lambda-1} r_i^{(k)}$.

Поскольку все $|r_i| < 1$, веса d_i убывают по модулю: $\sum|d_i^{(\lambda)}| \leq \sum|d_i^{(L)}| = D_L$. Следовательно, оценка $T_\lambda \leq \frac{2D_L}{1-D_L} = \tau_L < 1$ справедлива и для шага λ.

Вывод:

Последовательность $|s_\lambda - \rho_1|$ ограничена сверху геометрической прогрессией со знаменателем $\tau_L < 1$, следовательно $s_\lambda \to \rho_1$. Теорема доказана.

2.2. Доказательство глобальной сходимости

В предыдущем разделе мы доказали, что если итерация с переменным сдвигом (Стадия 3) запускается с "хорошим" начальным приближением (условия теоремы 3.1 выполнены), то она сходится. Теперь необходимо доказать, что первые две стадии алгоритма гарантированно приводят нас к такому состоянию.

Это ключевой момент доказательства "глобальной" сходимости: алгоритм работает для **любого** полинома, независимо от начального распределения нулей.

Теорема 3.2 (Глобальная сходимость).

Пусть фиксированный сдвиг s на второй стадии выбран так, что $|s - \rho_1| < |s - \rho_i|$ для всех $i = 2, \dots, j$. Тогда при достаточно большом L (количестве итераций второй стадии) начальные условия для третьей стадии будут выполнены, и последовательность s_λ сходится к ρ_1.

Доказательство:

Нам нужно показать, что при большом L выполняются три условия из Теоремы 3.1:

1. $|s_L - \rho_1| < \frac{1}{2}R$.
2. $c_1^{(L)} \neq 0$.
3. $D_L < \frac{1}{3}$.

Анализ коэффициентов после Стадии 2. Рассмотрим структуру полинома $H^{(L)}(z)$ после M шагов без сдвига и $L - M$ шагов с фиксированным сдвигом s. Согласно Лемме 1, коэффициенты $c_i^{(L)}$ имеют вид: $c_i^{(L)} = c_i^{(0)}(0 - \rho_i)^{-M}(s - \rho_i)^{-(L-M)} = c_i^{(0)}(-\rho_i)^{-M}(s - \rho_i)^{-(L-M)}$. Начальные коэффициенты $c_i^{(0)} = m_i$

(кратности корней), так как $H^{(0)} = P'$. Следовательно: $c_i^{(L)} = m_i(-\rho_i)^{-M}(s - \rho_i)^{-(L-M)}$.

Проверка условия 2 ($c_1^{(L)} \neq 0$). Так как $m_1 \neq 0$, $\rho_1 \neq 0$ (нули в начале координат удаляются предварительно), и $s \neq \rho_1$ (сдвиг выбирается случайным образом), то $c_1^{(L)}$ никогда не обращается в ноль.

Проверка условия 3 ($D_L < 1/3$). Рассмотрим отношение $D_L = \sum_{i=2}^{j} \left|\frac{c_i^{(L)}}{c_1^{(L)}}\right|$. Подставим явные формулы для коэффициентов: $\frac{c_i^{(L)}}{c_1^{(L)}} = \frac{m_i(-\rho_i)^{-M}(s-\rho_i)^{-(L-M)}}{m_1(-\rho_1)^{-M}(s-\rho_1)^{-(L-M)}} = \frac{m_i}{m_1}\left(\frac{\rho_1}{\rho_i}\right)^M \left(\frac{s-\rho_1}{s-\rho_i}\right)^{L-M}$. По условию выбора сдвига s, выполняется неравенство $|s - \rho_1| < |s - \rho_i|$ для всех $i \geq 2$. Обозначим $\gamma = \max_{i\geq 2}\left|\frac{s-\rho_1}{s-\rho_i}\right|$. Ясно, что $\gamma < 1$. Тогда: $D_L \leq \sum_{i=2}^{j}\left|\frac{m_i}{m_1}\left(\frac{\rho_1}{\rho_i}\right)^M\right| \gamma^{L-M}$. Зафиксируем M. При $L \to \infty$ множитель γ^{L-M} стремится к нулю. Следовательно, существует такое L_0, что для всех $L > L_0$ величина D_L станет меньше любого наперед заданного числа, в частности $1/3$.

Проверка условия 1 ($|s_L - \rho_1| < R/2$). Напомним, что s_L вычисляется как $s_L = s - \frac{P(s)}{\bar{H}^{(L)}(s)}$. Раскроем это выражение. $P(s) = \sum_{k=1}^{j} c_k^{(L)} P_k(s) \cdot \frac{1}{\sum c_k^{(L)} \ldots}$ (нет, используем более прямую связь). Используем тождество $P(s)/P_1(s) = s - \rho_1$. Тогда: $s_L - \rho_1 = (s - \rho_1) - \frac{P(s)}{\bar{H}^{(L)}(s)}$. Подставим выражение для $\bar{H}^{(L)}(s) = \frac{1}{\mathcal{K}_{L-1}}\sum c_i^{(L)} P_i(s)$ (с учетом нормировки). Более простое доказательство использует оценку относительной ошибки, аналогичную формуле (3.2), но для перехода от фиксированного сдвига к переменному. В статье Дженкинса-Трауба (1970) используется следующее неравенство (вытекающее из свойств Ньютона): $|s_L - \rho_1| \leq |s - \rho_1| \frac{2D_L}{1-D_L}$. Поскольку мы можем сделать D_L сколь угодно малым за счет увеличения L, то множитель $\frac{2D_L}{1-D_L}$ также стремится к нулю. Следовательно, $|s_L - \rho_1|$ может быть сделано меньше любой величины, в том числе $R/2$.

Таким образом, при достаточно большом L все условия сходимости выполнены.

2.3. Скорость сходимости

Одной из главных характеристик алгоритма Дженкинса-Трауба является его **сверхквадратичная** скорость сходимости на третьей стадии. В то время как

метод Ньютона имеет квадратичную сходимость (порядок 2), данный метод сходится быстрее.

Определим ошибку на шаге λ как $\epsilon_\lambda = |s_{L+\lambda} - \rho_1|$. Определим константу асимптотической ошибки $C(\lambda)$: $C(\lambda) = \frac{\epsilon_{\lambda+1}}{\epsilon_\lambda^2}$. Для классического Ньютона $C(\lambda) \approx$ const. Здесь же $C(\lambda) \to 0$.

Теорема 4.1 (R-порядок сходимости).

Для третьей стадии алгоритма выполняется оценка: $C(\lambda) \leq \frac{2}{R}\tau_L^{\lambda(\lambda-1)/2}$, где $\tau_L < 1$ — параметр сжатия из доказательства сходимости.

Доказательство:

Вернемся к уравнению эволюции ошибки (3.2): $\frac{s_{\lambda+1}-\rho_1}{s_\lambda-\rho_1} = \frac{\sum_{i=2}^{j} d_i^{(\lambda+1)}(r_i^{(\lambda)}-1)}{1+\sum_{i=2}^{j} d_i^{(\lambda+1)}}$. (Здесь мы упростили знаменатель дроби в числителе, так как $r_i^{(\lambda)} \approx 1$). Точнее, из формулы (4.4) статьи: $\frac{s_{\lambda+1}-\rho_1}{(s_\lambda-\rho_1)^2} = \frac{\sum_{i=2}^{j} \frac{d_i^{(\lambda+1)}}{s_\lambda-\rho_i}(r_i^{(\lambda)}-1)}{1+\cdots}$ Заметим, что $d_i^{(\lambda+1)}$ убывает очень быстро. Рекурсия для d_i: $d_i^{(\lambda+1)} = d_i^{(\lambda)} r_i^{(\lambda)} = d_i^{(L)} \prod_{k=L}^{\lambda} r_i^{(k)}$. Поскольку $|r_i^{(k)}| \leq \tau_L < 1$, произведение содержит λ сомножителей, каждый из которых мал. Более того, так как ошибка уменьшается квадратично, $r_i^{(k)}$ становится все меньше. Грубая оценка дает $|d_i^{(L+\lambda)}| \sim \tau_L^{\lambda^2/2}$ (так как произведение r накапливается). В теореме используется более консервативная оценка $\lambda(\lambda-1)/2$.

Это означает, что ошибка на следующем шаге $\epsilon_{\lambda+1}$ пропорциональна ϵ_λ^2, умноженному на фактор, который сам стремится к нулю. Это эквивалентно тому, что R-порядок сходимости (R-order) алгоритма равен:

- Для простых корней: не менее 2.618 (квадрат золотого сечения ϕ^2).
- В общем случае: быстрее 2.

Примечание: В статье Форда (1977) проведен более детальный анализ порядка сходимости для обобщенного класса методов и доказана оценка $1 + \phi \approx 2.618$ для всех методов типа Дженкинса-Трауба.

2.4. Алгоритм для действительных полиномов (Real Polynomials)

Для полиномов с вещественными коэффициентами комплексная арифметика является избыточной и вычислительно дорогой (одно комплексное умножение

= 4 вещественных, сложение = 2). Более того, вещественный алгоритм должен находить комплексно-сопряженные корни парами $(\rho, \bar{\rho})$ в виде квадратичного множителя $z^2 + uz + v$, оставаясь в поле вещественных чисел.

Для этого вводится новый класс полиномов — K**-полиномы**.

Определение K-полиномов:

Пусть $\sigma(z) = z^2 + uz + v = (z - s_1)(z - s_2)$ — вещественный квадратичный трехчлен, соответствующий паре сдвигов s_1, s_2 (которые могут быть комплексно-сопряженными). Рекурсия для K-полиномов определяется как "двойной шаг" H-рекурсии, выполненный в вещественной арифметике.

Определение (Fixed-shift K-polynomials):

$K^{(\lambda+1)}(z) = \frac{1}{\sigma(z)}\left[K^{(\lambda)}(z) - (Az + B)P(z)\right]$, где A и B — вещественные коэффициенты, выбираемые так, чтобы выражение в скобках делилось на $\sigma(z)$. Это условие эквивалентно тому, что выражение в скобках обращается в ноль в корнях s_1 и s_2.

Явная формула (аналог комплексного случая): $K^{(\lambda+1)}(z) = \frac{1}{\sigma(z)}\left[K^{(\lambda)}(z) - \left(\frac{K^{(\lambda)}(s_1)}{P(s_1)}\frac{z-s_2}{s_1-s_2} + \frac{K^{(\lambda)}(s_2)}{P(s_2)}\frac{z-s_1}{s_2-s_1}\right)P(z)\right]$.

<u>Сходимость к квадратичному фактору</u>

Если пара сдвигов (s_1, s_2) находится ближе к паре корней (ρ_1, ρ_2), чем к остальным, то последовательность нормированных K-полиномов сходится к полиному: $\frac{P(z)}{(z-\rho_1)(z-\rho_2)}$. Это позволяет извлечь квадратичный множитель.

На третьей стадии вещественного алгоритма используется **итерация с переменным квадратичным сдвигом**: $\sigma^{(\lambda+1)}(z) =$ аппроксимация квадратичного множителя на основе $K^{(\lambda+1)}$. Матричная интерпретация этого процесса (раздел 8 статьи Дженкинса-Трауба, 1970, Real) показывает, что это эквивалентно **квадратичной обобщенной итерации Рэлея** (Quadratic Generalized Rayleigh Iteration).

Часть 3. Реализация, модификации и эффективность метода

Теоретический фундамент алгоритма Дженкинса-Трауба, изложенный в предыдущих частях, требует тщательной адаптации при переносе в программный код. Реальные вычисления производятся в арифметике с плавающей точкой конечной точности, что создает риски переполнения, исчезновения порядка (underflow) и потери значимости. В этой части мы рассмотрим ключевые аспекты реализации, опираясь на классические программы CPOLY (для комплексных полиномов) и RPOLY (для вещественных), а также обсудим современные модификации, использующие быстрое преобразование Фурье (FFT), и методологию тестирования.

3.1. Вычислительные аспекты и реализация (CPOLY/RPOLY)

Программная реализация алгоритма должна решать ряд технических проблем, которые игнорируются в чисто математическом описании.

Масштабирование (Scaling)

Перед началом итераций необходимо масштабировать полином $P(z)$, чтобы избежать переполнения или исчезновения порядка при вычислении его значений. Если коэффициенты a_i слишком велики или малы, это может привести к ошибкам еще до начала поиска корней.

В реализации RPOLY используется следующий подход. Вычисляется параметр масштаба S, основанный на границах диапазона представимых чисел машины (пусть M_{min} и M_{max} — минимальное и максимальное положительные числа с плавающей точкой). Если $\max|a_i|$ слишком велико или мало, все коэффициенты умножаются на масштабный множитель $\sigma = \mathrm{Base}^L$, где Base — основание системы счисления (обычно 2 или 16), а L выбирается так, чтобы $\max|\sigma a_i| \approx 1$. Это операция выполняется без потери точности (только изменением экспоненты).

Границы корней (Cauchy bounds) и выбор начального сдвига

Для выбора начального сдвига s на второй стадии (Fixed-shift process) необходимо знать область локализации корней. Используется классическая

оценка Коши. Пусть $P(z) = z^n + a_1 z^{n-1} + \cdots + a_n$. Нижняя граница модулей корней β (т.е. $|\rho_i| \geq \beta$) определяется как единственный положительный корень вспомогательного полинома:

$$Q(z) = z^n + |a_1| z^{n-1} + \cdots + |a_{n-1}| z - |a_n| = 0.$$

Это уравнение легко решается методом Ньютона, так как функция $Q(z)$ монотонна для $z > 0$. Найденное значение β используется как радиус окружности, на которой выбирается начальный сдвиг s. Обычно s выбирается случайным образом на этой окружности: $s = \beta e^{i\phi}$, где ϕ — случайный угол. Это гарантирует, что $|s| \leq \min|\rho_i|$, и с высокой вероятностью s окажется ближе к самому малому корню ρ_1, чем к остальным.

Стабильность дефляции

После нахождения корня ρ_1 (или пары корней в RPOLY) полином $P(z)$ делится на $(z - \rho_1)$ (или квадратичный фактор). Полученный полином $Q(z)$ используется для поиска следующих корней. Этот процесс называется **дефляцией**. Ошибки округления при делении могут привести к тому, что корни $Q(z)$ будут существенно отличаться от оставшихся корней $P(z)$, если дефляция нестабильна. Как показал Уилкинсон, дефляция стабильна, если корни удаляются в порядке возрастания их модулей. Алгоритм Дженкинса-Трауба по своей природе находит корни именно в таком порядке (благодаря первой стадии без сдвига и выбору начального s на малой окружности). Это встроенное свойство обеспечивает высокую точность даже для полиномов высокой степени. Однако существуют патологические случаи (например, кластеры корней на разных радиусах), где порядок может нарушиться. Для контроля точности рекомендуется после нахождения всех корней выполнить одну итерацию метода Ньютона для каждого корня, используя исходный (недефлированный) полином $P(z)$.

Критерии остановки

Третья стадия алгоритма (Variable-shift) должна быть остановлена, когда приближение s_λ станет достаточно точным. Стандартный критерий $|s_{\lambda+1} - s_\lambda| < \epsilon$ ненадежен для кратных корней. В RPOLY используется более робастный критерий, основанный на анализе ошибки вычисления значения полинома (running error analysis). Пусть $\mathrm{fl}(P(s))$ — вычисленное значение полинома в машинной арифметике. Из-за ошибок округления оно отличается от истинного значения $P(s)$. Можно оценить границу этой ошибки $B(s)$, такую что $|P(s) -$

$\mathrm{fl}(P(s))| \leq B(s)$. Итерации прекращаются, когда вычисленное значение становится "неотличимым от нуля" на фоне шума округления:

$$|\mathrm{fl}(P(s_\lambda))| \leq B(s_\lambda).$$

Эта граница $B(s)$ вычисляется параллельно со схемой Горнера:

Если $P(z) = \sum a_i z^{n-i}$, то $P(s)$ вычисляется как $p_0 = a_0, p_k = sp_{k-1} + a_k$. Граница ошибки e_k рекурсивно растет:

$$e_k = |s|e_{k-1} + \epsilon(|sp_{k-1}| + |p_k|),$$

где ϵ — машинная точность. Финальное значение e_n и есть $B(s)$.

3.2. Применение Быстрого Преобразования Фурье (FFT)

Классический алгоритм Дженкинса-Трауба имеет сложность $O(n^2)$ для нахождения всех корней (так как каждый шаг требует $O(n)$ операций, а корней n). Для полиномов очень высокой степени (например, $n > 1000$) это может быть медленно. Узким местом является вычисление значений полинома в точках окружности на второй стадии для выбора лучшего сдвига.

<u>Модификация Хагера (Hager's Modification)</u>

В работе Хагера (1986) предложено использовать Быстрое Преобразование Фурье (FFT) для одновременного вычисления значений полинома $P(z)$ в множестве точек на окружности. Пусть нам нужно оценить $P(z)$ в L точках $z_k = \beta e^{i2\pi k/L}$ $(k = 0, \dots, L-1)$, лежащих на окружности радиуса β. Задача сводится к вычислению:

$$P(z_k) = \sum_{j=0}^{n} a_j \, \beta^{n-j} e^{i2\pi k(n-j)/L}.$$

Обозначим $\tilde{a}_j = a_j \beta^{n-j}$. Тогда вектор значений $P(z_k)$ есть дискретное преобразование Фурье последовательности коэффициентов $\tilde{a}_j$ (возможно, дополненной нулями до длины L). Использование FFT позволяет вычислить все L значений за время $O(L\log L)$, в то время как прямой метод требует $O(n \cdot L)$. Это позволяет проверить гораздо больше пробных точек на второй стадии и выбрать в качестве начального сдвига s ту точку, где $|P(z)|$ минимален. Это значительно повышает вероятность успешного старта третьей стадии и ускоряет глобальную сходимость.

3.3. Принципы тестирования программ

Создание надежного "черного ящика" для поиска корней требует тщательного тестирования. Дженкинс и Трауб сформулировали принципы тестирования, которые стали стандартом.

Классы тестовых полиномов

1. **Случайные полиномы:**
   - Случайные коэффициенты (обычно корни концентрируются около единичной окружности).
   - Случайные корни (равномерно распределенные в некоторой области).
2. **Плохо обусловленные полиномы:**
   - Полиномы Уилкинсона $W(z) = \prod_{k=1}^{n}(z-k)$.
   - Полиномы с большими коэффициентами (ортогональные полиномы, например, Чебышева или Лежандра).
3. **Полиномы с кластерами корней:**
   - Кратные корни $(z-1)^k$.
   - Близкие корни $(z-1)(z-1-\epsilon)$.
   - Кластеры вида $\rho(1 \pm \epsilon)$.
4. **Геометрические конфигурации:**
   - Корни на окружностях разного радиуса (проверка стабильности дефляции).
   - "Вертушки" и регулярные многоугольники.

Метрики качества

Для оценки качества работы программы используются следующие метрики:

- **Точность (Accuracy):**
  Сравнение найденных корней с известными точными значениями.
- **Обратная ошибка (Backward Error):**
  Если точные корни неизвестны, вычисляется норма невязки $P(\tilde{\rho})$ или восстанавливается полином $\tilde{P}(z)$ по найденным корням и сравнивается с исходным (норма коэффициентов $||P - \tilde{P}||$).
- **Надежность (Robustness):**
  Способность алгоритма не аварийно завершаться (overflow/underflow) и находить решение для патологических случаев.
- **Эффективность (Efficiency):**
  Время работы в зависимости от степени n (проверка асимптотики $O(n^2)$).

В ходе обширных тестов (как авторами, так и независимыми исследователями) алгоритм Дженкинса-Трауба показал высокую надежность и точность, превосходя многие альтернативы (например, метод Лагерра или метод сопровождающей матрицы с QR-алгоритмом) по скорости при сохранении сопоставимой точности.

Литература

1. Jenkins, M. A., and Traub, J. F. A three-stage variable-shift iteration for polynomial zeros and its relation to generalized Rayleigh iteration. Numerische Mathematik, 14, 252–263, 1970.
2. Jenkins, M. A., and Traub, J. F. A three-stage algorithm for real polynomials using quadratic iteration. SIAM Journal on Numerical Analysis, 7 (4), 545–566, 1970.
3. Jenkins, M. A., and Traub, J. F. Algorithm 419: Zeros of a Complex Polynomial. Communications of the ACM, 15 (2), 97–99, 1972.
4. Jenkins, M. A. Algorithm 493: Zeros of a Real Polynomial. ACM Transactions on Mathematical Software, 1 (2), 178–189, 1975.
5. Jenkins, M. A., and Traub, J. F. Principles for Testing Polynomial Zerofinding Programs. ACM Transactions on Mathematical Software, 1 (1), 26–34, 1975.
6. Ford, J. A. A Generalization of the Jenkins-Traub Method. Mathematics of Computation, 31 (137), 193–203, 1977.
7. Hager, W. W. A Modified Fast Fourier Transform for Polynomial Evaluation and the Jenkins-Traub Algorithm. Numerische Mathematik, 50, 253–261, 1987.
8. Pederson, R. N. Newton's Method and the Jenkins-Traub Algorithm. Proceedings of the American Mathematical Society, 97 (4), 687–690, 1986.